Relatório Técnico

MyKeeper – Refatoração com Padrões de Projeto GoF

Design de Software · Avaliação Somativa RA2 · Bacharelado em Engenharia de Software · PUCPR

1. Identificação e contextualização (4.1)

Equipe	[inserir nome da equipe]
Integrantes	Lucas, Rodrigo, Giovanna e Emilly [completar com sobrenomes]
Domínio	Gestão doméstica de estoque de alimentos, receitas, compras e suporte ao usuário
Problema	Famílias perdem controle do que têm em casa, do que comprar e de quanto os preços variam; o sistema centraliza cadastro, estoque, listas de compras, metas de consumo e histórico de preços.
Público-alvo	Usuários domésticos (controle da casa) e equipe de suporte (tickets e contas).
Projeto de origem	MyKeeper, desenvolvido anteriormente na disciplina de projeto em equipe (versão web PHP/MySQL). Esta entrega parte da versão Java/JavaFX do mesmo sistema (10 CRUDs com persistência em arquivo) e a refatora e evolui.

2. Descrição da solução (4.2)

O sistema é uma aplicação desktop em Java 21 com JavaFX. O menu principal abre dez módulos CRUD: **Produtos, Categorias, Estoque, Fornecedores, Receitas, Lista de Compras, Meta de Consumo, Histórico de Preços, Contas de Suporte e Tickets de Suporte**. Os dados são persistidos em arquivos `.dat` por serialização.

Situação inicial. Os dez módulos eram implementados por cópia: dez classes de arquivo praticamente idênticas e dez telas com o mesmo fluxo repetido (cerca de 3.000 linhas), com o status de ticket, lista e estoque como texto livre e sem qualquer regra de fluxo.

Evolução realizada nesta entrega (além da refatoração):

- **Fluxo de status** com regras reais: ticket (Aberto → Em andamento → Resolvido → Fechado, com reabertura), lista de compras (Em elaboração → Finalizada → Comprada) e estoque (Ativo → Inativo → Arquivado). Cada estado define o que pode ser editado ou excluído; as regras são aplicadas também na camada de dados.
- **Ordenação escolhida pelo usuário** em todas as listagens (por ID, nome, categoria, data, preço, tempo, limite etc.).
- **Alertas de variação de preço**: ao registrar um preço, o sistema avisa aumentos acima de 10% e novos menores preços do produto, e grava um log de auditoria (`alertas_preco.log`).
- **Testes automatizados** (JUnit, 13 casos) cobrindo os quatro padrões.

Como executar: JDK 21 e `./mvnw clean javafx:run` (testes: `./mvnw test`). As instruções completas estão no README do projeto.

3. Padrões utilizados (4.3)

Foram aplicados quatro padrões GoF, todos comportamentais, cada um resolvendo um problema concreto do código original. A tabela resume; as subseções detalham, com trechos reais do código-fonte.

Padrão	Problema de design identificado	Onde entra
Template Method	Dez repositórios e dez telas com o mesmo algoritmo copiado	RepositorioArquivo, CrudUI (e especializações)
State	Status como texto livre, sem regras de transição, edição ou exclusão	Ticket, Lista de Compras e Estoque
Strategy	Nenhum critério de ordenação; adicionar critérios exigiria if/switch nas telas	Listagens de todas as telas
Observer	Variação de preço não era percebida; acoplar alertas ao cadastro seria rígido	Histórico de Preços

3.1 Template Method

Intenção	Definir o esqueleto de um algoritmo em uma classe base, deixando que subclasses redefinam passos específicos sem alterar a estrutura do algoritmo.
Problema	As classes *ArquivoXxx* (10 cópias) repetiam ler, salvar, gerar ID, adicionar, atualizar e deletar; as telas *XxxUI* (10 cópias) repetiam montagem, seleção na tabela, tratamento dos botões e mensagens. Só mudavam o tipo, o nome do arquivo e os campos.
Por que este padrão	A sequência de passos é idêntica e o que varia é pequeno e bem delimitado, o cenário típico do padrão. Herança com métodos gancho é mais simples aqui do que composição, pois cada tela/repositório é de fato um caso particular do mesmo fluxo.
Classes envolvidas	**Repositórios:** RepositorioArquivo<T> (AbstractClass), RepositorioComEstado<T>, ArquivoProduto, ArquivoHistoricoPreco, ArquivoTicketSuporte e demais (ConcreteClass); Identificavel. **Telas:** CrudUI<T> (AbstractClass), CrudComEstadoUI<T>, ProdutoUI, TicketSuporteUI, HistoricoPrecosUI e demais.
Como foi aplicado	Os métodos adicionar/atualizar/deletar (repositório) e adicionar/atualizar/deletar/limpar/carregarTabela/montar (tela) são *final* e fixam a ordem dos passos. O que varia são passos abstratos (caminhoArquivo, titulo, repositorio, camposFormulario, lerFormulario...) e ganchos opcionais com implementação padrão vazia (validar, validarAtualizacao, validarRemocao, aposAdicionar, podeEditar, podeExcluir, botoesExtras). Há dois níveis: RepositorioComEstado e CrudComEstadoUI especializam o esqueleto para entidades com State.
Benefícios	Cada repositório simples passou a ter cerca de 10 linhas (só o caminho do arquivo) e cada tela só declara seus campos. Corrigir um bug do fluxo ou criar um novo módulo passa a ser uma alteração em um único lugar.

model/RepositorioArquivo.java – método template (linhas 57-63)

```
public final void adicionar(T novo) {
    validar(novo);                      // 1. gancho
    ArrayList<T> lista = lerLista();    // 2. passo fixo
    lista.add(novo);                    // 3. passo fixo
    salvarLista(lista);                 // 4. passo fixo
    aposAdicionar(novo);                // 5. gancho
}
```

model/ArquivoProduto.java – subclasse concreta (só o passo variável) (linhas 4-8)

```
public class ArquivoProduto extends RepositorioArquivo<Produto> {

    public static final ArquivoProduto INSTANCE = new ArquivoProduto();

    private ArquivoProduto() { }
```

ui/CrudUI.java – fluxo fixo da tela (linhas 162-172)

```
private void adicionar() {
    try {
        T novo = lerFormulario(repositorio().gerarId());
        repositorio().adicionar(novo);
        mensagem(nomeEntidade() + ": registro adicionado.");
```

```
        limpar();
        carregarTabela();
    } catch (RuntimeException ex) {
        mensagem(ex.getMessage());
    }
}
```

Template Method - repositórios e telas CRUD

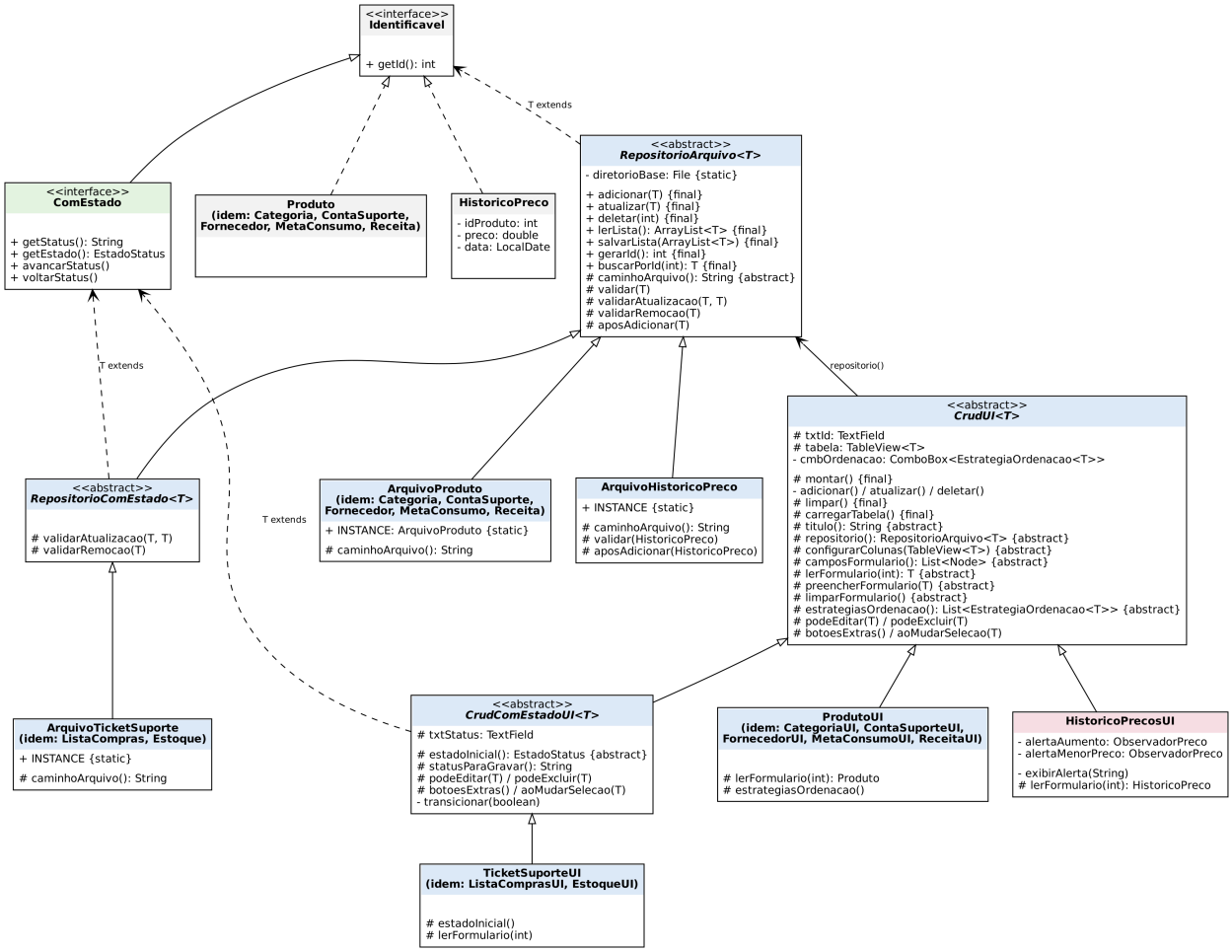


Figura 1 – Template Method: repositórios e telas CRUD.

3.2 State

Intenção	Permitir que um objeto altere seu comportamento quando seu estado interno muda, encapsulando cada estado em uma classe.
Problema	Ticket, lista e estoque guardavam o status como String livre: nada impedia um ticket de ir de Aberto direto a Fechado, ser excluído em atendimento ou editado depois de resolvido, e cada regra teria de virar if/switch espalhado pelas telas.
Por que este padrão	O comportamento (pode editar? pode excluir? qual o próximo passo?) depende do estado e muda conforme o objeto evolui, o que caracteriza State. O fluxo é finito, pequeno e estável, e as regras são diferentes em cada estado.
Classes envolvidas	**Context:** TicketSuporte, ListaCompras, Estoque (via interface ComEstado). **State:** EstadoStatus. **ConcreteState:** TicketAberto, TicketEmAndamento, TicketResolvido, TicketFechado; ListaEmElaboracao, ListaFinalizada, ListaComprada; EstoqueAtivo, EstoqueInativo, EstoqueArquivado. Apoio: EstadosTicket/EstadosLista/EstadosEstoque (reconstroem o estado a partir do nome gravado) e RepositorioComEstado.
Como foi aplicado	O contexto mantém o atributo estado e delega a transição: avancarStatus() faz estado = estado.proximo(). Cada ConcreteState informa proximo(), anterior(), permiteEdicao() e permiteExclusao(). O nome do status continua sendo gravado como texto (compatível com os .dat antigos) e o objeto de estado, marcado transient, é reconstruído ao ler. A tela habilita ou desabilita botões consultando o estado; o repositório rejeita edições, exclusões e transições inválidas com IllegalStateException, de modo que a regra vale mesmo sem a interface.

Benefícios	As regras de negócio ficam junto de cada estado, sem condicionais por texto de status. Incluir um novo estado (ex.: Cancelado) é criar uma classe e ajustar a transição vizinha. A mesma estrutura atende três entidades.

model/estado/TicketEmAndamento.java – ConcreteState (linhas 4-5)

```
public class TicketEmAndamento implements EstadoStatus {
    @Override public String getNome() { return "Em andamento"; }
```

model/TicketSuporte.java – Context delega ao estado (linhas 67-70)

```
public void avancarStatus() {
    estado = getEstado().proximo();   // delegação ao objeto de estado
    status = estado.getNome();
}
```

model/RepositorioComEstado.java – regras do estado no repositório (linhas 14-35)

```
protected void validarAtualizacao(T atual, T novo) {
    String nomeAtual = atual.getEstado().getNome();
    String nomeNovo = novo.getStatus();
    if (nomeAtual.equalsIgnoreCase(nomeNovo)) {
        // mesmo estado: é uma edição de dados
        if (!atual.getEstado().permiteEdicao()) {
            throw new IllegalStateException("Registro no estado \"" + nomeAtual + "\" não pode ser editado.");
        }
        return;
    }
    // estado diferente: só são aceitas as transições previstas pelo próprio estado
    boolean avancou = atual.getEstado().permiteAvancar()
            && atual.getEstado().proximo().getNome().equalsIgnoreCase(nomeNovo);
    boolean voltou = atual.getEstado().permiteVoltar()
            && atual.getEstado().anterior().getNome().equalsIgnoreCase(nomeNovo);
    if (!avancou && !voltou) {
        throw new IllegalStateException("Transição inválida de \"" + nomeAtual + "\" para \"" + nomeNovo + "\".");
    }
}

@Override
protected void validarRemocao(T atual) {
```

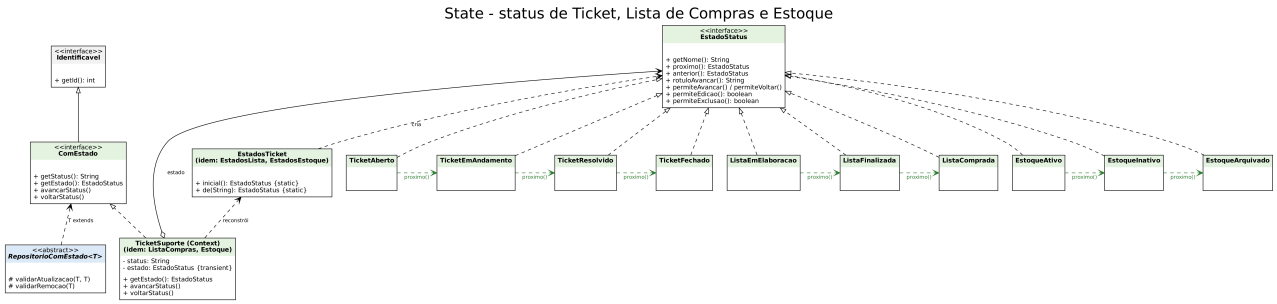


Figura 2 – State: contexto, interface de estado e estados concretos (setas verdes: próximo estado).

3.3 Strategy

Intenção	Definir uma família de algoritmos, encapsular cada um e torná-los intercambiáveis, de modo que variem independentemente de quem os usa.
Problema	As listagens apareciam sempre na ordem do arquivo. Oferecer vários critérios por tela com if/switch no código de cada uma duplicaria a lógica nas dez telas e exigiria alterar todas a cada novo critério.
Por que este padrão	Há uma família de algoritmos (critérios de ordenação) com a mesma interface, escolhida em tempo de execução pelo usuário. Strategy é a forma direta de trocar o critério sem condicionais.
Classes envolvidas	**Strategy:** EstrategiaOrdenacao<T>. **Base:** OrdenacaoPorComparador<T>. **ConcreteStrategy:** OrdenacaoPorId, OrdenacaoPorTexto, OrdenacaoPorNumero, OrdenacaoPorData. **Context:** CrudUI<T>.
Como foi aplicado	O ComboBox de cada tela é preenchido com as estratégias fornecidas pelo método estrategiasOrdenacao() (passo do Template Method). Ao escolher outra, CrudUI guarda a nova estratégia e recarrega a tabela chamando estrategia.ordenar(lista). As estratégias devolvem uma cópia ordenada; a lista original não é alterada. A de texto ignora acentos e maiúsculas (Collator pt-BR).

Benefícios	Novos critérios entram como novas classes (ou novas instâncias parametrizadas) sem tocar nas telas; a lógica de ordenação é testável isoladamente, sem JavaFX.

ordenacao/EstrategiaOrdenacao.java (linhas 12-19)

```
public interface EstrategiaOrdenacao<T> {

    /** Texto exibido na lista de critérios da tela. */
    String getNome();

    /** Devolve uma nova lista ordenada (a original não é alterada). */
    List<T> ordenar(List<T> itens);
}
```

ui/CrudUI.java – Context aplica a estratégia escolhida (linhas 213-217)

```
protected final void carregarTabela() {
    ArrayList<T> lista = repositorio().lerLista();
    EstrategiaOrdenacao<T> estrategia = cmbOrdenacao.getValue();
    dados.setAll(estrategia == null ? lista : estrategia.ordenar(lista));
}
```

ui/ProdutoUI.java – critérios oferecidos (linhas 48-55)

```
    protected List<EstrategiaOrdenacao<Produto>> estrategiasOrdenacao() {
        return List.of(
                new OrdenacaoPorId<>(true), new OrdenacaoPorId<>(false),
                new OrdenacaoPorTexto<Produto>("Nome", Produto::getNome, true),
                new OrdenacaoPorTexto<Produto>("Nome", Produto::getNome, false),
                new OrdenacaoPorTexto<Produto>("Categoria", Produto::getCategoria, true));
    }
}
```

Strategy - critérios de ordenação das listagens

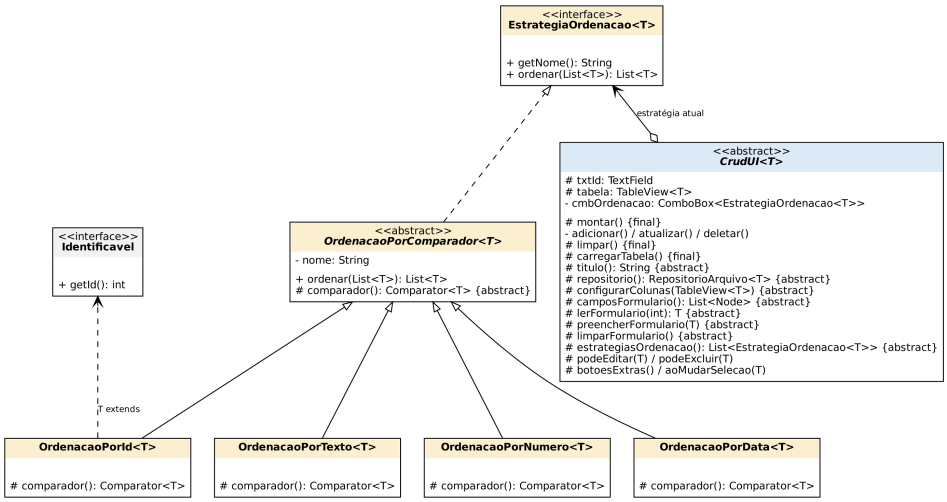


Figura 3 – Strategy: critérios de ordenação.

3.4 Observer

Intenção	Definir uma dependência um-para-muitos entre objetos, de modo que, quando um muda de estado, todos os dependentes sejam notificados e atualizados automaticamente.
Problema	O histórico de preços registrava valores, mas ninguém percebia aumentos ou boas oportunidades. Colocar essa lógica dentro do cadastro (ou da tela) acoplaria o repositório a cada novo tipo de alerta e a classes de interface.
Por que este padrão	Quem registra o preço não deve saber quem se interessa por ele, e o conjunto de interessados muda (tela aberta ou fechada, log sempre ativo, novos alertas). É o cenário clássico de publicação/assinatura.
Classes envolvidas	**Subject:** MonitorPrecos. **Observer:** ObservadorPreco. **ConcreteObserver:** AlertaAumentoPreco, AlertaMenorPreco, RegistroAlertasLog. **Evento:** VariacaoPreco. **Publicador:** ArquivoHistoricoPreco (gancho aposAdicionar do Template Method). **Assinantes:** HistoricoPrecosUI e Main.

Como foi aplicado	Ao adicionar um preço, ArquivoHistoricoPreco calcula o preço anterior e o menor preço já registrado do produto e chama MonitorPrecos.notificar(variacao). A tela de histórico registra os dois alertas ao abrir e os remove ao fechar; a Main registra o log durante toda a execução. Uma falha em um observador não impede os demais de serem avisados.
Benefícios	O repositório não conhece os alertas concretos; novos observadores (e-mail, push, integração com a lista de compras) são adicionados sem alterar quem publica. O Observer reaproveita o gancho do Template Method, mostrando a composição dos padrões.

observer/MonitorPrecos.java – Subject (linhas 32-40)

```
public void notificar(VariacaoPreco variacao) {
    for (ObservadorPreco o : observadores) {
        try {
            o.precoRegistrado(variacao);
        } catch (RuntimeException e) {
            // um observador com defeito não pode impedir os demais de serem avisados
            System.err.println("Falha em observador de preço: " + e.getMessage());
        }
    }
```

model/ArquivoHistoricoPreco.java – publicação do evento (linhas 30-44)

```
protected void aposAdicionar(HistoricoPreco novo) {
    HistoricoPreco anterior = null;
    Double menor = null;
    for (HistoricoPreco h : lerLista()) {
        if (h.getId() == novo.getId() || h.getIdProduto() != novo.getIdProduto()) continue;
        if (menor == null || h.getPreco() < menor) menor = h.getPreco();
        boolean maisRecente = anterior == null
                || h.getData().isAfter(anterior.getData())
                || (h.getData().isEqual(anterior.getData()) && h.getId() > anterior.getId());
        if (!h.getData().isAfter(novo.getData()) && maisRecente) anterior = h;
    }
    MonitorPrecos.INSTANCE.notificar(new VariacaoPreco(
            novo.getIdProduto(),
            anterior == null ? null : anterior.getPreco(),
            menor, novo.getPreco(), novo.getData()));
```

ui/HistoricoPrecosUI.java – assinatura e cancelamento (linhas 29-38)

```
public HistoricoPrecosUI() {
    txtData.setPromptText("Data");
    montar();
    MonitorPrecos.INSTANCE.registrar(alertaAumento);
    MonitorPrecos.INSTANCE.registrar(alertaMenorPreco);
    setOnHidden(e -> {
        MonitorPrecos.INSTANCE.remover(alertaAumento);
        MonitorPrecos.INSTANCE.remover(alertaMenorPreco);
    });
}
```

Observer - alertas de variação de preço

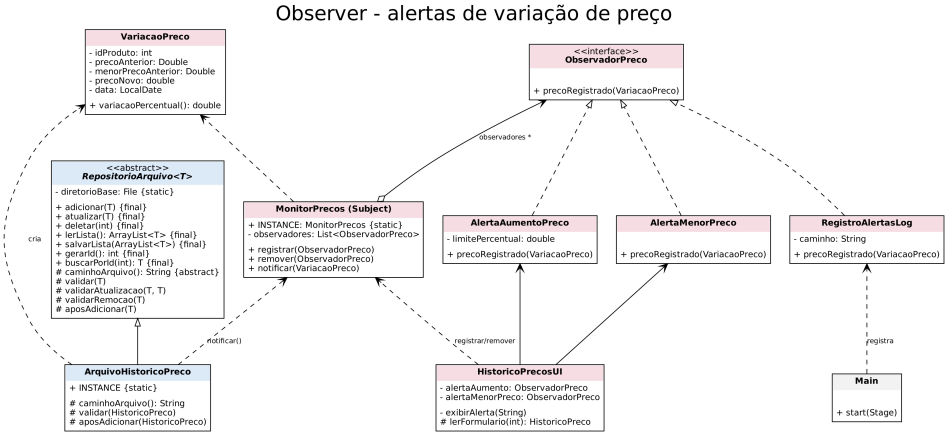


Figura 4 – Observer: alertas de variação de preço.

3.5 Interação entre os padrões

Os padrões se complementam: o Template Method fornece os pontos de extensão (passos e ganchos) usados pelos demais. O Strategy entra como um passo da tela (estrategiasOrdenacao), o State como especialização do esqueleto (RepositorioComEstado, CrudComEstadoUI) e o Observer como um gancho (aposAdicionar). Por isso a refatoração em Template Method foi feita primeiro.

4. Diagrama de classes completo

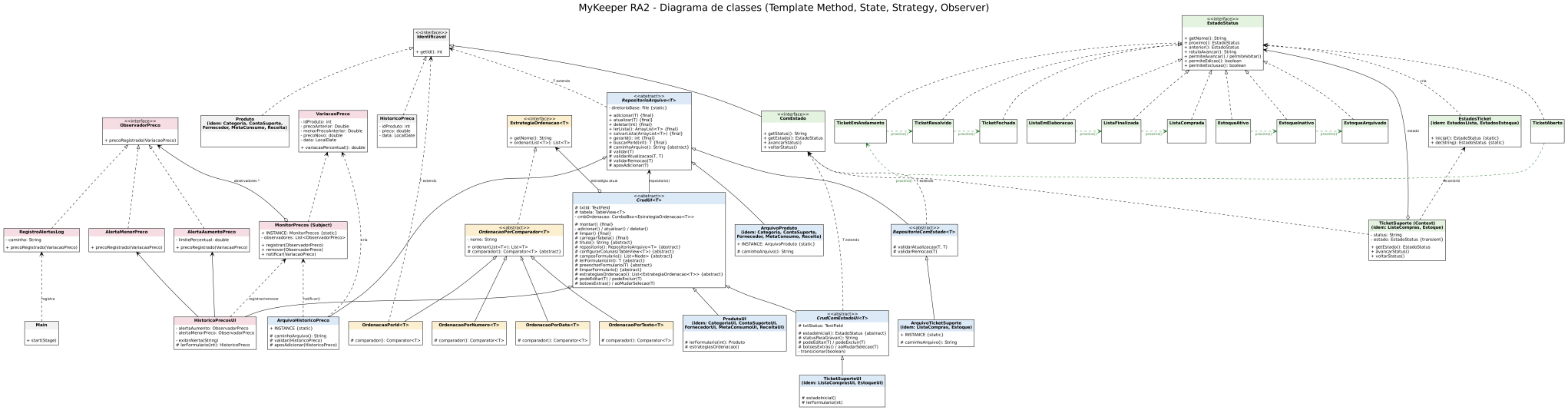


Figura 5 – Diagrama de classes completo. Versão em alta resolução em docs/diagramas/diagrama_completo.png (também .svg e .pdf vetoriais). Legenda de cores: azul = Template Method; verde = State; amarelo = Strategy; rosa = Observer; cinza = entidades e Main. Setas: triângulo vazado = herança; tracejada com triângulo = implementação; tracejada simples = dependência; losango = agregação.

5. Análise de aplicabilidade dos padrões GoF (4.4)

Os 4 padrões aplicados estão descritos na seção 3. Abaixo, os demais 19 padrões do catálogo GoF e o motivo, no contexto do MyKeeper, de não serem aplicados. Considerou-se pertinente apenas o padrão que resolve um problema existente no sistema, evitando aplicação artificial.

Padrão	Tipo	Por que não se aplica ao MyKeeper
Singleton	Criacional	Os repositórios têm construtor privado e um campo INSTANCE e o MonitorPrecos tem uma instância compartilhada, mas é só um meio de compartilhar o mesmo objeto, sem um problema de design próprio (não há recurso caro nem ciclo de vida a controlar). Por isso não foi contado como padrão avaliado.
Factory Method	Criacional	As telas são criadas diretamente no menu e há uma única implementação de cada. Não há família de produtos escolhida em tempo de execução. O Estados* é uma fábrica estática simples de reconstrução, não o Factory Method do catálogo.
Abstract Factory	Criacional	Não existem famílias de objetos relacionados que precisem variar juntas (ex.: temas de interface ou bancos diferentes).
Builder	Criacional	As entidades têm no máximo 6 atributos e construtores diretos; não há construção em etapas nem muitas variações opcionais.
Prototype	Criacional	Não há objetos caros de criar nem necessidade de clonar instâncias em tempo de execução.
Adapter	Estrutural	Não há integração com biblioteca ou serviço de interface incompatível; JavaFX e serialização são usados diretamente.
Bridge	Estrutural	Há uma única forma de persistência (arquivo serializado). Se o sistema passasse a suportar também banco de dados, separar abstração e implementação do repositório seria pertinente.
Composite	Estrutural	Não há estruturas em árvore ou todo-parte tratadas uniformemente; ingredientes de receita são texto simples.
Decorator	Estrutural	Não há comportamentos opcionais empilhados dinamicamente sobre os mesmos objetos.
Facade	Estrutural	Os módulos são independentes e o menu apenas abre telas; não há subsistema complexo que exija interface simplificada.
Flyweight	Estrutural	O volume de dados é pequeno (cadastros domésticos); não há grande número de objetos com estado compartilhável.
Proxy	Estrutural	Acesso a arquivo local, sem objeto remoto, carga preguiçosa custosa ou controle de acesso por perfil.
Chain of Responsibility	Comportamental	A validação de formulários é uma sequência fixa e curta dentro de cada tela; não há cadeia de tratadores configurável em tempo de execução.
Command	Comportamental	Não há requisito de desfazer/refazer, fila ou registro de ações; as ações dos botões são diretas.
Interpreter	Comportamental	O sistema não define linguagem ou gramática a interpretar.
Iterator	Comportamental	A iteração sobre listas já é fornecida pelas coleções Java; criar iterador próprio não traria benefício.
Mediator	Comportamental	As telas não conversam entre si; a comunicação indireta que existe (preço → alertas) é resolvida pelo Observer.
Memento	Comportamental	Sem requisito de salvar/restaurar estados anteriores de objetos (undo).
Visitor	Comportamental	Não há hierarquia heterogênea de elementos sobre a qual aplicar operações novas com frequência.

Nota: o enunciado pede análise dos padrões estudados em sala; foram cobertos os 23 do catálogo GoF. Se algum não foi trabalhado em aula, a linha correspondente pode ser removida.

6. Reflexão e lições aprendidas (4.5)

Rascunho a ser revisado pela equipe antes da entrega: o texto abaixo descreve decisões e dificuldades técnicas reais do desenvolvimento e deve ser complementado com a experiência dos integrantes.

Dificuldades e decisões

- **Chamada de método abstrato no construtor.** A primeira ideia era montar a tela dentro do construtor de CrudUI. Isso falha porque os campos da subclasse ainda não foram inicializados. A solução foi o método montar(), chamado como última instrução do construtor de cada tela.
- **State e serialização.** Os arquivos .dat já guardavam o status como texto. Em vez de serializar os objetos de estado, mantivemos o texto e tornamos o objeto transient, reconstruído por uma fábrica; dados antigos com texto livre voltam ao estado inicial.
- **Onde aplicar as regras do State.** Validar só na interface deixaria o modelo vulnerável. As regras (editar, excluir, transições legais) foram levadas ao repositório por um gancho do Template Method, e a tela apenas as espelha nos botões.
- **Strategy como classes ou lambdas.** Uma lista de Comparators resolveria o problema, mas esconderia o padrão. Optamos por classes de estratégia parametrizadas, mantendo a interface e o contexto explícitos, ao custo de um pouco mais de código.
- **Observer e ciclo de vida.** Registrar observadores na abertura de uma tela e esquecer de removê-los os acumularia a cada abertura. A tela passou a se desregistrar ao ser fechada.
- **Herança versus composição.** O Template Method com herança resolveu bem a duplicação, mas acopla as telas à classe base. Em um sistema maior, uma estratégia de formulário por composição poderia ser preferível.
- **Limitação dos testes.** Os testes automatizados cobrem modelo, estados, ordenação e alertas. As telas JavaFX foram verificadas apenas em compilação e devem ser conferidas manualmente.

Aprendizados

- Identificar a variabilidade antes de escolher o padrão: o ponto de partida foi mapear o que se repetia e o que mudava.
- Padrões se combinam: o Template Method abriu os pontos de extensão que o State, o Strategy e o Observer usaram.
- O ganho é mensurável: de dez repositórios e dez telas copiados para uma base comum, com módulos novos criados em dezenas de linhas.
- Um padrão só se justifica quando há um problema real; por isso a seção 5 explica por que 19 padrões ficaram de fora.